

Webex Contact Center MCP Server

Natural-language administration for enterprise contact centers.

This project bridges [Claude Code](#) and the official Webex Contact Center (WxCC) Admin APIs. Ask for what you want in plain English, and Claude reads or changes your tenant's live configuration on your behalf — no portal clicking, no memorized endpoint paths.

What it covers

21 skills route natural-language requests to **12 MCP tools** spanning **14 config entities**:

Domain	Entities
People & routing	Users, Teams, Skills, Skill Profiles
Reachability	Entry Points, Dial Numbers, Address Books, Outdial ANIs
Queuing	Queues (Contact Service Queues), Sites
Agent experience	Desktop Profiles, Desktop Layouts, Auxiliary Codes, Multimedia Profiles

Plus **GraphQL aggregation** for reporting — grouped counts and sums over calls, tasks, and agent sessions without pulling raw records — and full **webhook** CRUD for event subscriptions.

The safety model

Writes are mechanically gated, not politely requested:

- **Dry run by default.** A write call without `confirm` writes nothing — it returns the exact field-level diff (or the object that would be destroyed) and the rollback plan.
- **Every confirmed write re-reads and diffs.** This API can return `200` while silently ignoring a field; the tool reports that rather than trusting the status code.
- **Deletes pre-flight references** through the API's own `incoming-references` check, and refuse with the blocker list instead of letting you discover a 412 afterward.
- **Every result names its tenant** — pulled from the tenant's own record, not a configured label — and is tagged `[PRODUCTION]` or `[trial/sandbox]`.

Enterprise-grade security

- **Zero stored credentials.** Deployed to Cloud Run, the server holds no Webex token at all — each caller runs their own OAuth flow, and the token arrives on the request and is forgotten.
- **Your own identity, your own audit trail.** Every operation runs under the caller's personal Webex sign-in, inheriting their exact RBAC rather than a shared service account's.
- **No switch-tenant command.** Each tenant is bound to its own dedicated MCP server; acting on the wrong tenant means calling a differently-named tool, not flipping a setting.
- **Org-pinned by URL.** A cloud connection can declare the org id it expects; a token from any other org is rejected with `403 wrong_tenant` on every request, not just at login.

Honest limits

Every capability above was run against a live tenant before it was written down. What isn't there yet is listed here, not glossed over.

- Users are read/update only — creation and deletion stay in Control Hub.
- Phone numbers can't be invented; a dial-number record maps a number already in the Webex Calling inventory.
- Flows are out of scope by design. Cisco ships its own `flow-store` MCP server for flow authoring; this project owns the config flows bind to — entry points, queues, teams, skill profiles.
- Only the `us1` region has been exercised so far.